



Parallel & Distributed Computing

Lecture 3:

General Parallelism Terms

Farhad M. Riaz

Farhad.Muhammad@numl.edu.pk

Department of Computer Science
NUML, Islamabad

Some General Parallel Terminology

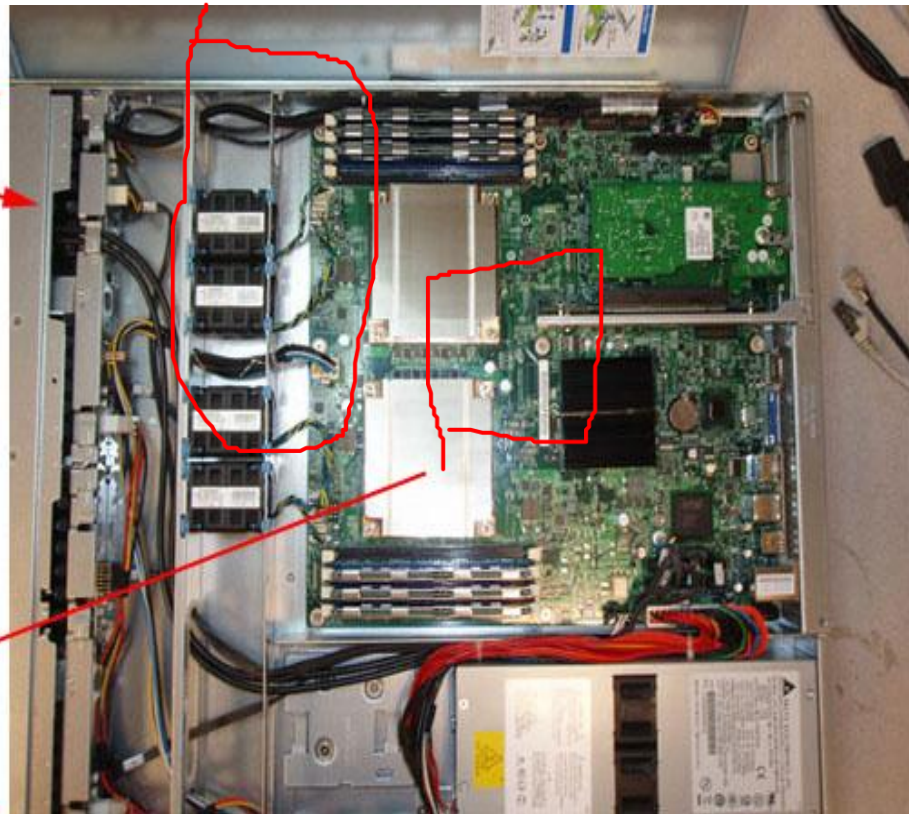
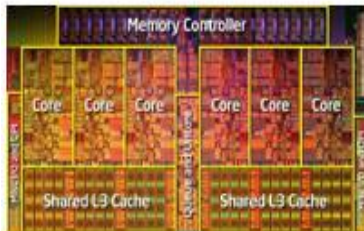
- **Supercomputing / High Performance Computing (HPC)**
 - Using the world's fastest and largest computers to solve large problems
- **Node**
 - A standalone "computer in a box". Usually comprised of multiple CPUs/processors/cores, memory, network interfaces, etc. Nodes are networked together to comprise a supercomputer.
- **CPU / Socket / Processor / Core**
 - In the past, a CPU (Central Processing Unit) was a singular execution component
 - Then, multiple CPUs were incorporated into a node
 - Then, individual CPUs were subdivided into multiple "cores", each being a unique execution unit
 - CPUs with multiple cores are sometimes called "sockets"
 - The result is a node with multiple CPUs, each containing multiple cores



Supercomputer - each blue light is a node

Node - standalone
Von Neumann computer

CPU / Processor / Socket - each
has multiple cores / processors.



Some General Parallel Terminology

- **Task**
 - A logically discrete section of computational work. A task is typically a program or program-like set of instructions that is executed by a processor. A parallel program consists of multiple tasks running on multiple processors.
- **Pipelining**
 - Breaking a task into steps performed by different processor units, with inputs streaming through, much like an assembly line; a type of parallel computing.
- **Shared Memory**
 - From a strictly hardware point of view, describes a computer architecture where all processors have direct (usually bus based) access to common physical memory. In a programming sense, it describes a model where parallel tasks all have the same "picture" of memory and can directly address and access the same logical memory locations regardless of where the physical memory actually exists.

Some General Parallel Terminology

- **Symmetric Multi-Processor (SMP)**
 - Shared memory hardware architecture where multiple processors share a single address space and have equal access to all resources.
- **Distributed Memory**
 - In hardware, refers to network based memory access for physical memory that is not common. As a programming model, tasks can only logically "see" local machine memory and must use communications to access memory on other machines where other tasks are executing.
- **Communications**
 - Parallel tasks typically need to exchange data. There are several ways this can be accomplished, such as through a shared memory bus or over a network, however the actual event of data exchange is commonly referred to as communications regardless of the method employed

Some General Parallel Terminology

- **Synchronization**

- The coordination of parallel tasks in real time, very often associated with communications.
- Often implemented by establishing a synchronization point within an application where a task may not proceed further until another task(s) reaches the same or logically equivalent point.
- Synchronization usually involves waiting by at least one task and can therefore cause a parallel application's wall clock execution time to increase.

- **Granularity**

- granularity is a qualitative measure of the ratio of computation to communication
- **Coarse:** relatively large amounts of computational work are done between communication events
- **Fine:** relatively small amounts of computational work are done between communication events

- **Observed Speedup**

- Observed speedup of a code which has been parallelized, defined as:
$$\frac{\text{wall-clock time of serial execution}}{\text{wall-clock time of parallel execution}}$$
- One of the simplest and most widely used indicators for a parallel program's performance.

Some General Parallel Terminology

- **Parallel Overhead**

- The amount of time required to coordinate parallel tasks, as opposed to doing useful work. Parallel overhead can include factors such as Task start-up time
 - Synchronizations
 - Data communications
 - Software overhead imposed by parallel languages, libraries, operating system, etc.
 - Task termination time

- **Massively Parallel**

- Refers to the hardware that comprises a given parallel system - having many processing elements. The meaning of "many" keeps increasing, but currently, the largest parallel computers are comprised of processing elements numbering in the hundreds of thousands

- **Embarrassingly Parallel**

- Solving many similar, but independent tasks simultaneously; little to no need for coordination between the tasks

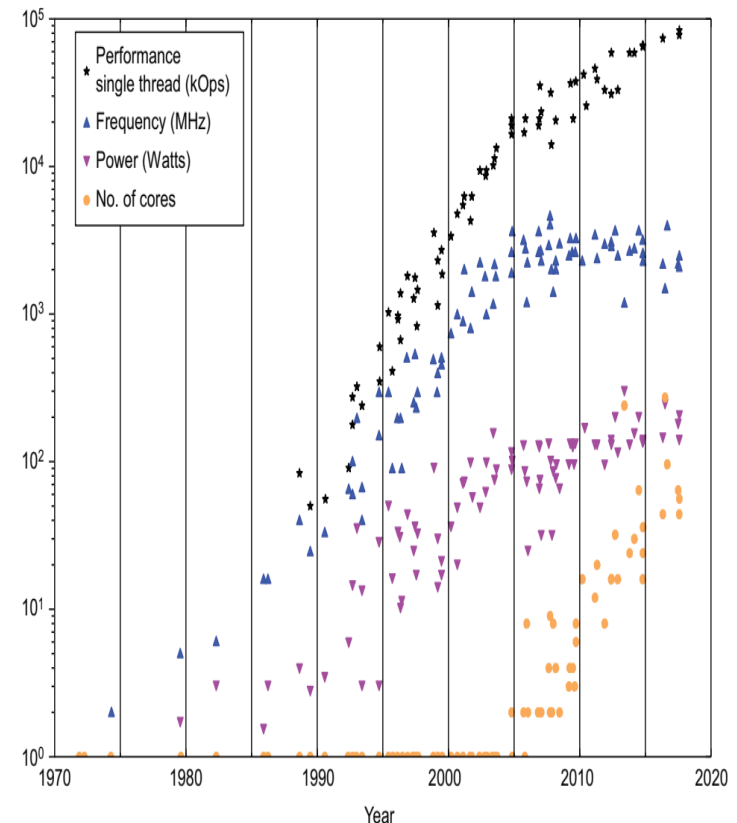
Some General Parallel Terminology

- **Scalability**

- Refers to a parallel system's (hardware and/or software) ability to demonstrate a proportionate increase in parallel speedup with the addition of more resources. Factors that contribute to scalability include:
 - Hardware - particularly memory-CPU bandwidths and network communication properties
 - Application algorithm
 - Parallel overhead related
 - Characteristics of your specific application

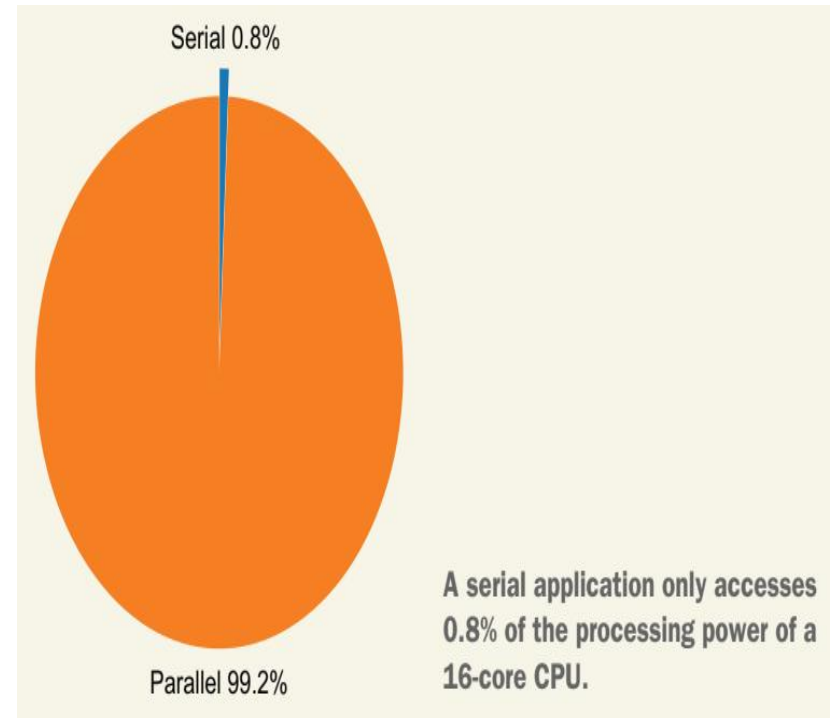
Why Parallel Computing is necessary?

- Rise of multi-core computing machines
- Under-utilization of resources
- Hyperthreading, introduced by Intel



Example

- Let's take a 16-core CPU with hyperthreading and a 256 bit-wide vector unit, commonly found in home desktops. A serial program using a single core and no vectorization only uses 0.8% of the theoretical processing capability of this processor.
- The calculation is
 - $16 \text{ cores} \times 2 \text{ hyperthreads} \times (256 \text{ bit-wide vector unit}) / (64\text{-bit double}) = 128\text{-way parallelism}$
where $1 \text{ serial path} / 128 \text{ parallel paths} = .008 \text{ or } 0.8\%$. The following figure shows that this is a small fraction of the total CPU processing power.



Benefits of Parallel Computing

- Lesser execution time with multiple cores
- Solve larger and complex problems
- Energy efficient system
- Cost reduction

FLOPS

- Floating point operations per second
- Measure of theoretical peak performance

$$\text{FLOPS} = \text{sockets} \times \frac{\text{cores}}{\text{socket}} \times \frac{\text{cycles}}{\text{second}} \times \frac{\text{FLOPs}}{\text{cycle}}$$

FLOPS

- Servers are the only computers that sometimes have more than one socket; for most home computers (desktop or laptop), “sockets” will be 1.
- Cores per socket depends on your CPU. It could be 2 (dual-core), 3, 4 (quad-core), 6 (hexa-core), or 8. There are some prototype CPUs with as many as 80 cores.
- “Clock cycles per second” refers to the speed of your CPU. Most modern CPUs are rated in gigahertz. So 2 GHz would be 2,000,000,000 clock cycles per second.
- The number of FLOPs per cycle also depends on the CPU. One of the fastest (home computer) CPUs is the Intel Core i7-970, capable of 4 double-precision or 8 single-precision floating-point operations per cycle.

Test

- Intel Core i7–970 has 6 cores. If it is running at 3.46 GHz and can perform 8 floating point operations per second, calculate the theoretical compute power of this machine.

Example

- Intel Core i7–970 has 6 cores. If it is running at 3.46 GHz, the formula would be:
- $1 \text{ (socket)} * 6 \text{ (cores)} * 3,460,000,000 \text{ (cycles per second)} * 8 \text{ (single-precision FLOPs per second)} = 166,080,000,000 \text{ single-precision FLOPs per second or } 83,040,000,000 \text{ double-precision FLOPs per second.}$
- 109 GFLOPS.

Amdahl's Law

- Theoretical speedup calculations

$$S_{\text{latency}}(s) = \frac{1}{(1 - p) + \frac{p}{s}}$$

where

S_{latency} is the theoretical speedup of the execution of the whole task;
 s is the speedup of the part of the task that benefits from improved system resources;

p is the proportion of execution time that the part benefiting from improved resources originally occupied.

Example 1

- If 30% of the execution time may be the subject of a speedup, p will be 0.3; if the improvement makes the affected part twice as fast, s will be 2. Amdahl's law states that the overall speedup of applying the improvement will be?

Example 2

- Assume that we are given a serial task which is split into four consecutive parts, whose percentages of execution time are $p_1 = 0.11$, $p_2 = 0.18$, $p_3 = 0.23$, and $p_4 = 0.48$ respectively. Then we are told that the 1st part is not sped up, so $s_1 = 1$, while the 2nd part is sped up 5 times, so $s_2 = 5$, the 3rd part is sped up 20 times, so $s_3 = 20$, and the 4th part is sped up 1.6 times, so $s_4 = 1.6$. By using Amdahl's law, the overall speedup is?

Amdahl's Law

Two independent parts **A** **B**

Original process

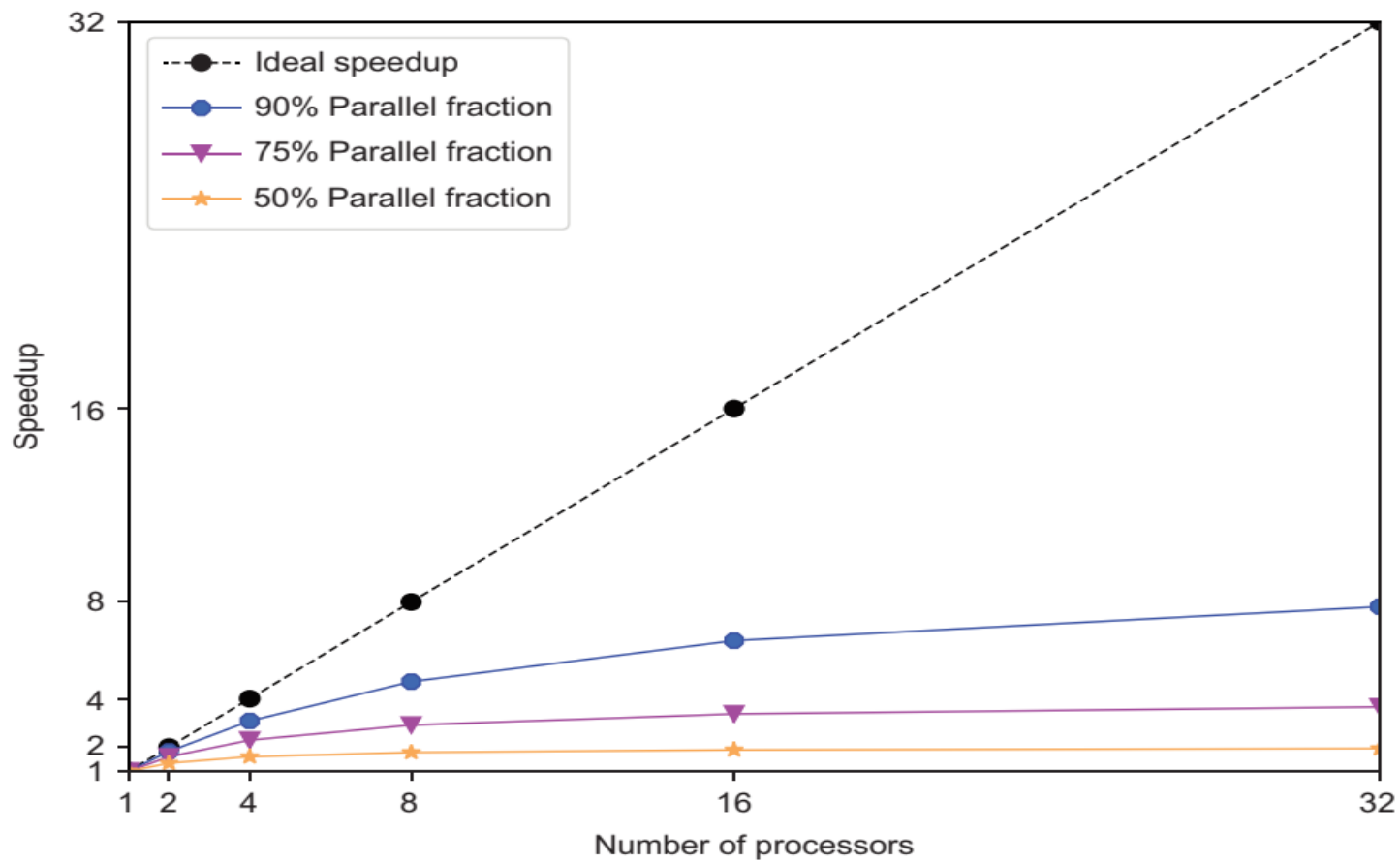


Make **B** 5x faster



Make **A** 2x faster





Amdahl's Law

